TECHNICAL REPORT

on

AI CHATBOT DEVELOPMENT PROJECT

Submitted in partial fulfilment of the requirements of the Internship Program

Submitted by

Sanjivani Pandurang Jadhav

B.Tech, Information Technology

Under the Mentorship of

Dr. Ritu Jain G, U2U Technology Team

USHA MITTAL INSTITUTE OF TECHNOLOGY

SNDT Women's University, Mumbai

Department of Information Technology

Internship Duration: 1 June 2026 – 15 July 2026

Group Number: 4

Certificate

This is to certify that Ms. Sanjivani Pandurang Jadhav, a student of B.Tech (Information Technology) at Usha Mittal Institute of Technology, SNDT Women's University, Mumbai, has successfully completed the internship project titled “AI Chatbot Development” under the mentorship of Dr. Ritu Jain G, U2U Technology Team, during the period 1 June 2026 to 15 July 2026.

During this internship, the intern demonstrated a sound understanding of client-server architecture, AI model integration, API development and testing, version control, and cloud deployment. The work carried out and the report submitted are found to be satisfactory and in partial fulfilment of the requirements of the internship program.

Internship Mentor

Dr. Ritu Jain G
U2U Technology Team

Internship Coordinator

Declaration

I, Sanjivani Pandurang Jadhav, hereby declare that this technical report titled “AI Chatbot Development Project” is a record of original work carried out by me during my internship, under the mentorship of Dr. Ritu Jain G, U2U Technology Team, between 1 June 2026 and 15 July 2026.

I further declare that this report, or any part of it, has not been submitted elsewhere for the award of any degree, diploma, or certificate, and that all sources of information used in this report have been duly acknowledged.

Sanjivani Pandurang Jadhav

B.Tech, Information Technology

Usha Mittal Institute of Technology, SNDT Women's University, Mumbai

Acknowledgement

I would like to express my sincere gratitude to my mentor, Dr. Ritu Jain G of the U2U Technology Team, for her valuable guidance, continuous support, and encouragement throughout the course of this internship. Her insights and feedback were instrumental in the successful completion of this project.

I am also thankful to Usha Mittal Institute of Technology, SNDT Women's University, Mumbai, for providing me with the opportunity to undertake this internship and for the support extended by the Department of Information Technology.

Finally, I would like to thank my family, friends, and everyone who supported and encouraged me during the course of this internship and the preparation of this report.

Sanjivani Pandurang Jadhav

Abstract

This report presents the development of an AI-powered chatbot application built using a client-server architecture. The system enables users to submit prompts through a web-based frontend interface; these prompts are transmitted to a Node.js and Express.js backend server, which communicates with the Google Gemini AI model through the Google GenAI SDK to generate intelligent responses that are then displayed to the user.

The project covers the complete development lifecycle, including frontend design, backend API development, AI model integration, API testing using PowerShell and cURL, error handling, version control using Git and GitHub, and cloud deployment using Render. The report also documents the system architecture, API specifications, testing outcomes, challenges encountered, and the results achieved.

The internship provided practical, hands-on experience in modern web development and AI integration practices, and the resulting application demonstrates a functional, deployable AI chatbot with a clear roadmap for future enhancement.

Table of Contents

List of Figures

1. Introduction

1.1 Project Details

Project Name: AI Chatbot Development

Student Name: Sanjivani Pandurang Jadhav

Branch: Information Technology

Institution: Usha Mittal Institute of Technology, SNDT Women's University, Mumbai

Internship Duration: 1 June 2026 – 15 July 2026

Group Number: 4

Mentor: Dr. Ritu Jain G, U2U Technology Team

1.2 Project Objective

The objective of this project is to develop an AI-powered chatbot application using a client-server architecture.

The system allows users to enter prompts through a frontend interface. The prompts are sent to a backend server, which processes them through the Google Gemini AI model. The generated AI response is then returned to the frontend and displayed to the user.

The project also focuses on API development, client-server communication, AI model integration, API testing, version control, documentation, and cloud deployment.

1.3 Project Overview

The AI Chatbot Development Project is a web-based application that enables users to interact with an artificial intelligence model through a simple chatbot interface.

The application consists of a frontend client and a backend server. The frontend provides an interface for entering user prompts and displaying AI-generated responses. The backend receives requests from the frontend and communicates with the Google Gemini AI model.

The project demonstrates the development of an AI-powered web application using modern web technologies and cloud deployment tools.

2. Technologies Used

The following technologies and tools were used during the development of the project:

- HTML
- CSS
- JavaScript
- Node.js
- Express.js
- Google Gemini AI
- Google GenAI SDK
- Git

- GitHub
- Render
- PowerShell
- cURL

3. System Architecture

The AI chatbot follows a client-server architecture consisting of the following major layers:

3.1 Client Layer

The Client Layer provides the user-facing chatbot interface. It allows users to:

- Enter a prompt.
- Submit the prompt.
- View the AI-generated response.
- Receive error messages when required.

The client layer is developed using:

- HTML
- CSS
- JavaScript

3.2 Server Layer

The Server Layer is developed using Node.js and Express.js. It performs the following functions:

- Receives requests from the frontend.
- Parses JSON request data.
- Validates user prompts.
- Communicates with the Google Gemini AI model.
- Returns AI-generated responses to the frontend.
- Provides supporting API endpoints.

3.3 AI Model Layer

The AI Model Layer provides the intelligence of the chatbot. The Google Gemini AI model processes the user's prompt and generates an appropriate response.

The project uses the Google GenAI SDK to communicate with the Gemini AI model.

3.4 Database Layer

A persistent database is not currently implemented in the project. The database layer is considered a future component that can be used to store:

- Conversation history.
- User information.
- User feedback.

- Other application data.

3.5 Architecture Flow

The basic system flow is illustrated below:

```
User
  |
  v
Client Layer (HTML, CSS, JavaScript)
  |
  v
POST /api/chat
  |
  v
Backend Server (Node.js, Express.js)
  |
  v
Google Gemini AI Model
  |
  v
AI-Generated Response
  |
  v
Backend Server -> Client Layer -> User
```

4. Implementation

4.1 Frontend Implementation

The frontend is implemented using HTML, CSS, and JavaScript. The frontend interface provides:

- A text input field.
- A submit button.
- A loading indicator.
- A response display area.

When the user enters a prompt and submits it, JavaScript collects the input and sends it to the backend using the Fetch API. The request is sent to:

```
POST /api/chat
```

The request contains the following JSON data:

```
{
  "prompt": "Hello"
}
```

After the backend processes the request and receives the AI-generated response, the response is displayed on the frontend interface.

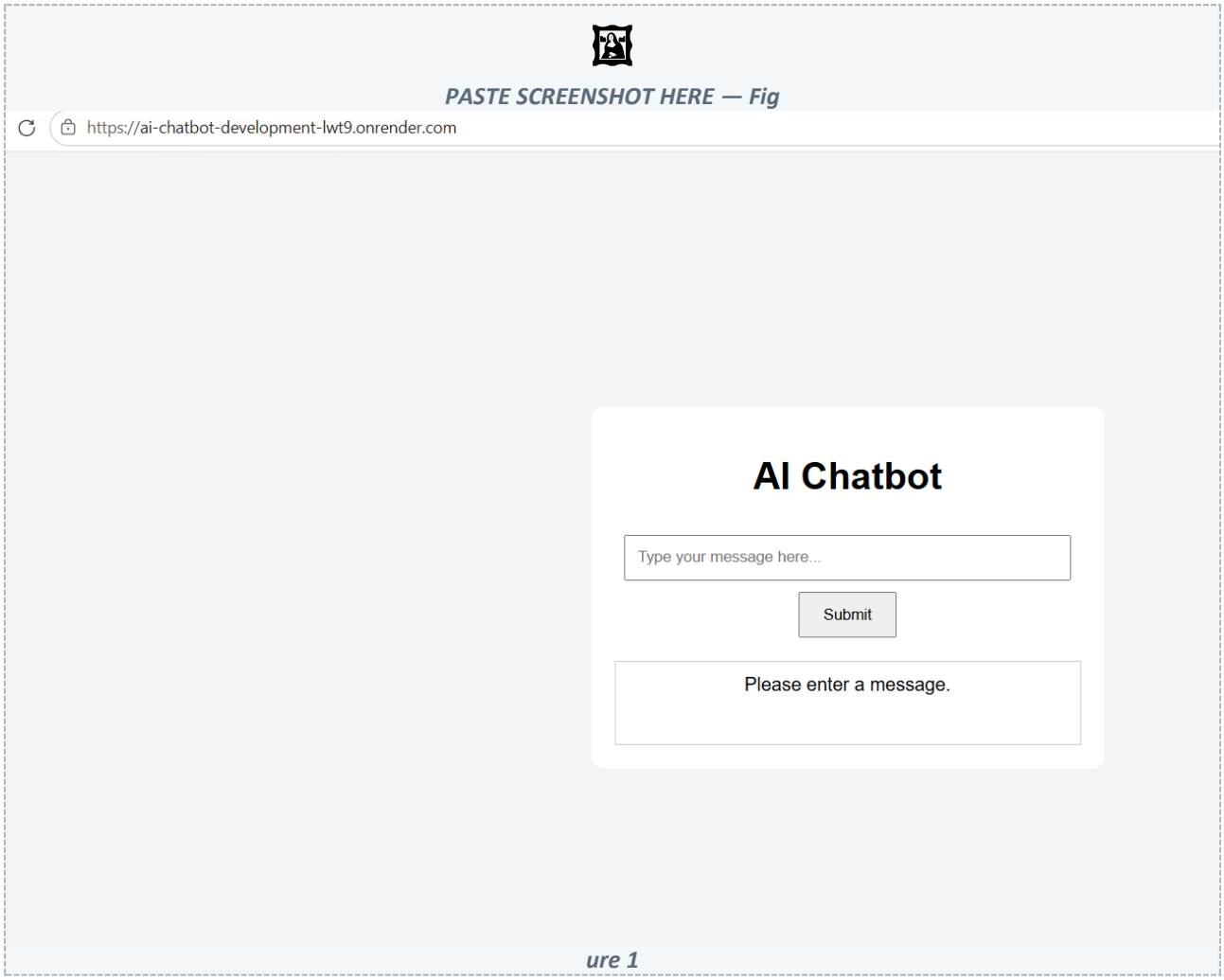

Figure 1: Chatbot frontend interface showing the input field, submit button and AI response area

4.2 Backend Implementation

The backend is developed using Node.js and Express.js. The backend performs the following operations:

1. Receives HTTP requests from the frontend.
2. Parses JSON request data.
3. Validates the user prompt.
4. Sends the prompt to the Google Gemini AI model.
5. Receives the generated response.
6. Returns the response to the frontend.

The server uses the environment-provided port during deployment:

```
const PORT = process.env.PORT || 3000;
```

This configuration allows the application to run both locally and on cloud deployment platforms.

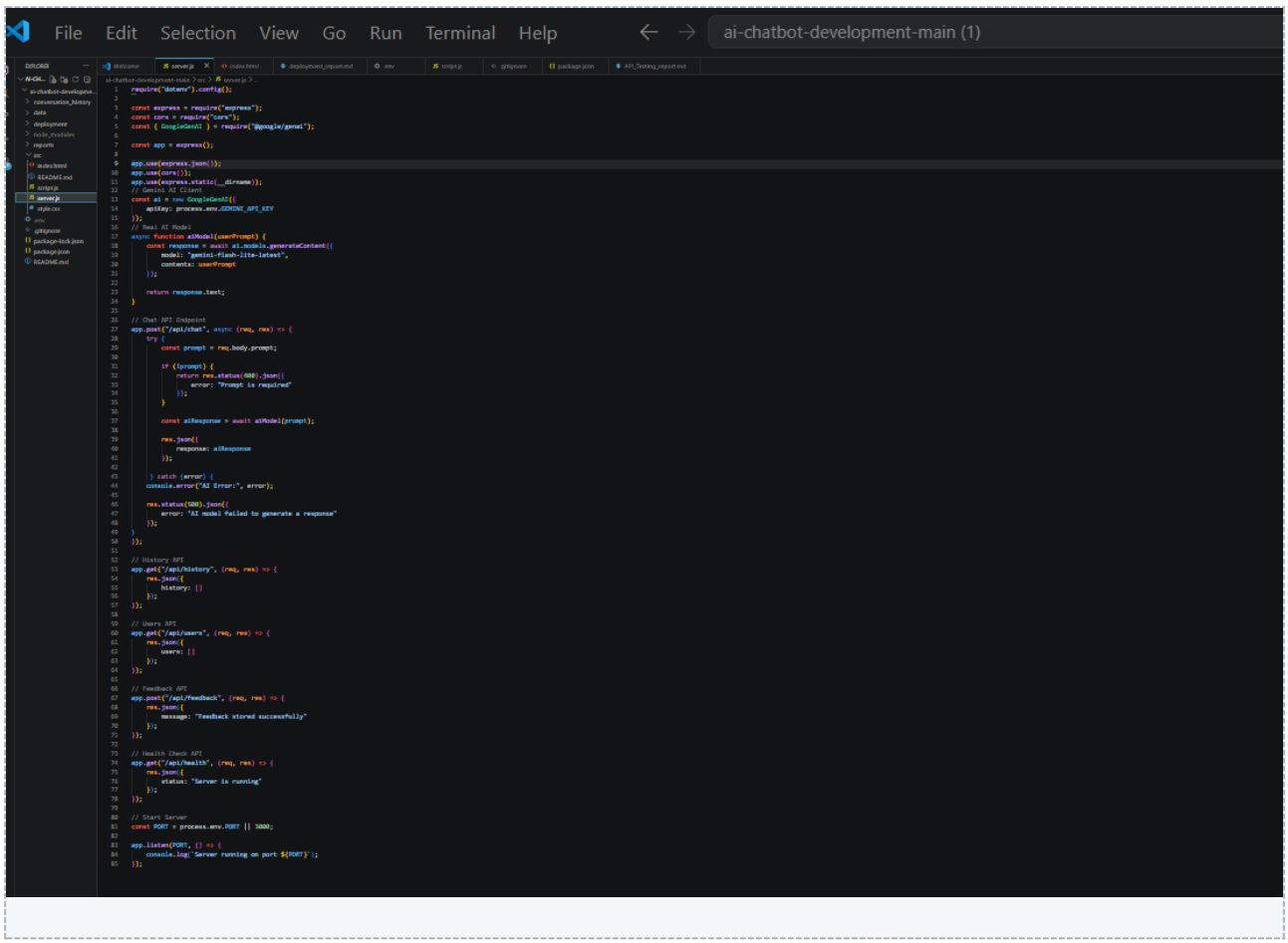

Figure 2: Backend server code (Node.js / Express.js) shown in the code editor

4.3 Google Gemini AI Integration

The project integrates Google Gemini AI using the Google GenAI SDK. The backend sends the user's prompt to the AI model for processing. The selected model is:

```
gemini-flash-lite-latest
```

The processing flow is:

```
User Prompt -> Backend Server -> Google Gemini AI
  -> Generated AI Response -> Backend Server -> Frontend
```

The generated response is returned by the backend and displayed to the user through the frontend interface.

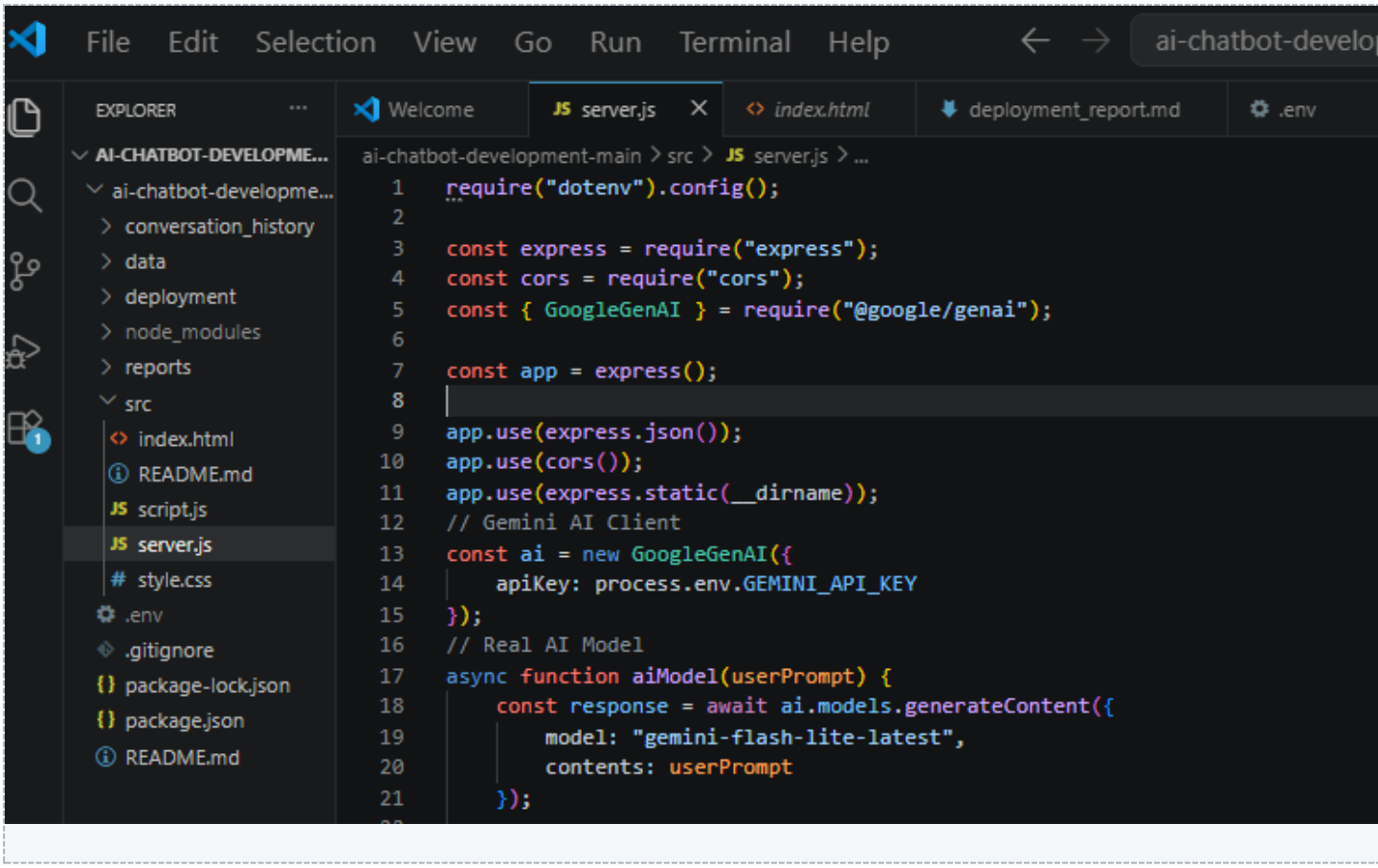

Figure 3: Google Gemini AI / Google GenAI SDK integration code snippet

5. API Design

5.1 API Endpoints

The backend provides the following API endpoints:

Endpoint	Method	Purpose
/api/chat	POST	Send prompts to the AI chatbot and receive AI-generated responses.
/api/history	GET	Provide an endpoint for future conversation history functionality.
/api/users	GET	Provide an endpoint for future user-related functionality.
/api/feedback	POST	Provide an endpoint for future user feedback functionality.
/api/health	GET	Check whether the API server is running properly.

5.2 Chat API

Endpoint:

```
POST /api/chat
```

Purpose: Receives a user prompt, sends it to the Google Gemini AI model, and returns the generated AI response.

Request:

```
{
  "prompt": "Explain artificial intelligence"
}
```

Successful Response:

```
{
  "response": "Artificial intelligence is..."
}
```

Error Response (if no prompt is provided):

```
{
  "error": "Prompt is required"
}
```

5.3 History API

Endpoint:

```
GET /api/history
```

Purpose: Provides an endpoint for future conversation history functionality.

Response:

```
{
  "history": []
}
```

Persistent conversation storage has not been implemented in the current version.

5.4 Users API

Endpoint:

```
GET /api/users
```

Purpose: Provides an endpoint for future user-related functionality.

Response:

```
{
  "users": []
}
```

User management and authentication have not been implemented in the current version.

5.5 Feedback API

Endpoint:

```
POST /api/feedback
```

Purpose: Provides an endpoint for future user feedback functionality.

Response:

```
{
  "message": "Feedback stored successfully"
}
```

Persistent feedback storage has not been implemented in the current version.

5.6 Health Check API

Endpoint:

```
GET /api/health
```

Purpose: Checks whether the API server is running properly.

Response:

```
{
  "status": "Server is running"
}
```

6. Client-Server Communication

The frontend communicates with the backend using the JavaScript Fetch API. The communication process is:

7. The user enters a prompt in the frontend interface.
8. The frontend sends the prompt to the backend.
9. The backend receives and validates the prompt.
10. The backend sends the prompt to the Google Gemini AI model.
11. The AI model generates a response.
12. The response is returned to the backend.
13. The backend sends the response to the frontend.
14. The frontend displays the AI-generated response.

The communication flow is illustrated below:

```
User -> Frontend Client -> POST /api/chat -> Backend Server
  -> Google Gemini AI Model -> Generated AI Response
  -> Backend Server -> Frontend Client -> User
```

7. API Testing

The backend APIs were tested to verify their functionality, response behaviour, status codes, and error handling. The testing was performed using:

- PowerShell Invoke-RestMethod
- cURL

The following functionalities were tested:

- Server health check.
- Chat API.
- AI-generated response handling.
- History endpoint.
- Users endpoint.
- Feedback endpoint.
- Empty prompt validation.
- Invalid route handling.
- HTTP status code verification.

7.1 Health Check Test

Request:

```
GET /api/health
```

Response:

```
{
  "status": "Server is running"
}
```

The health check API successfully returned a 200 OK response.

7.2 Chat API Test

Request:

```
{
  "prompt": "Explain artificial intelligence in one simple sentence"
}
```

The API successfully returned an AI-generated response.

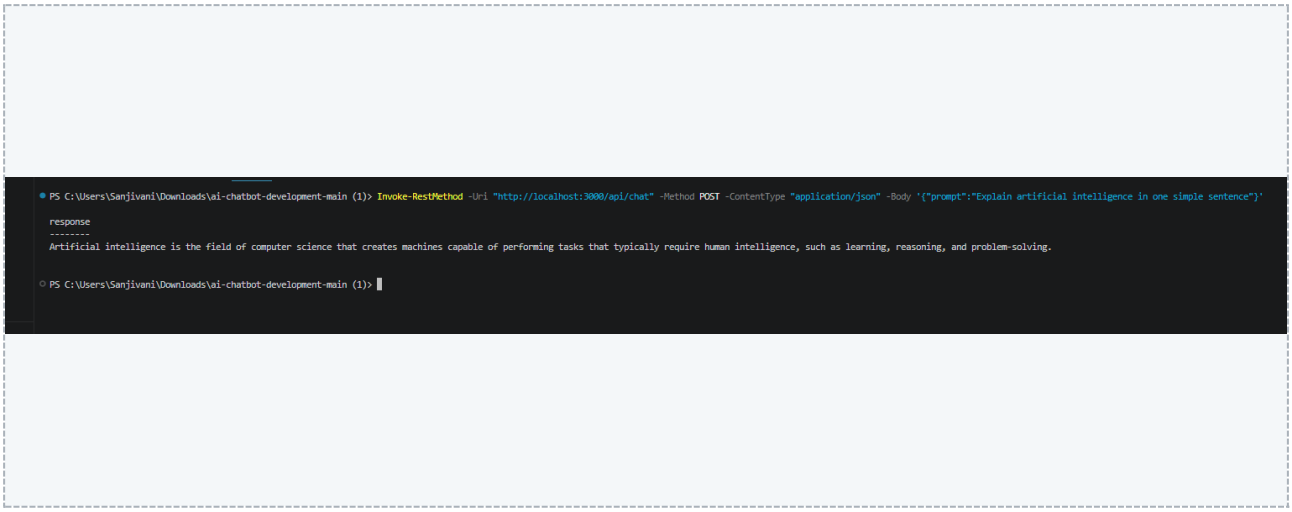

Figure 4: Chat API test executed using PowerShell / cURL, showing the AI-generated response

7.3 Empty Prompt Test

An empty request was sent to the chat endpoint:

```
{}
```

The server correctly rejected the request and returned:

```
{
  "error": "Prompt is required"
}
```

The request was returned with a 400 Bad Request status.

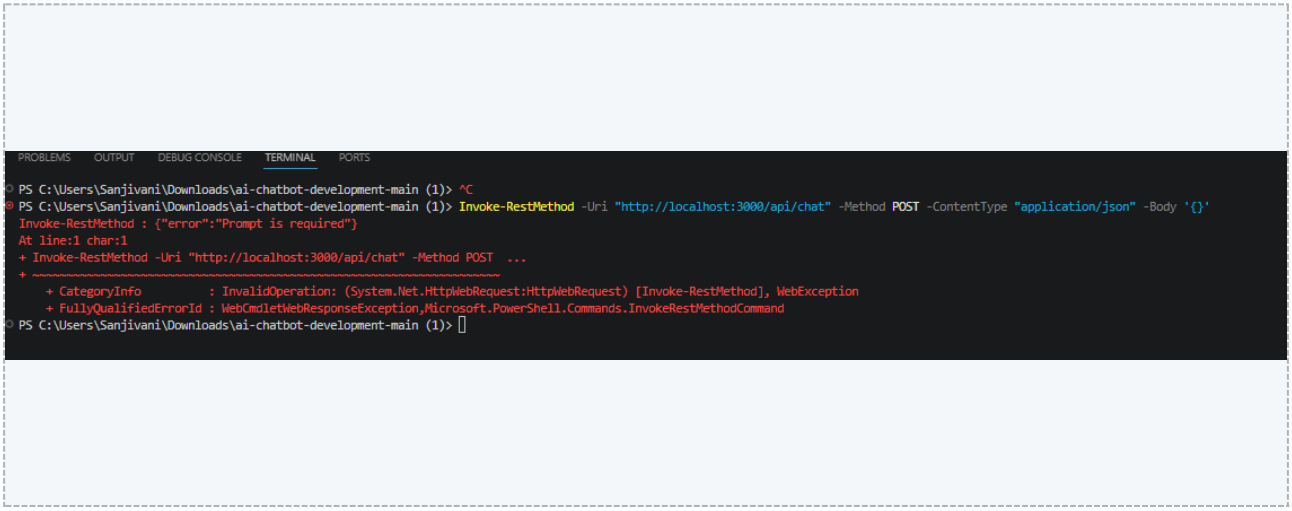

Figure 5: Empty prompt validation test result (400 Bad Request)

7.4 Invalid Endpoint Test

A request was sent to a non-existent endpoint:

```
GET /api/invalid
```

The server returned:

```
Cannot GET /api/invalid
```

This confirmed that invalid routes are not supported by the backend.

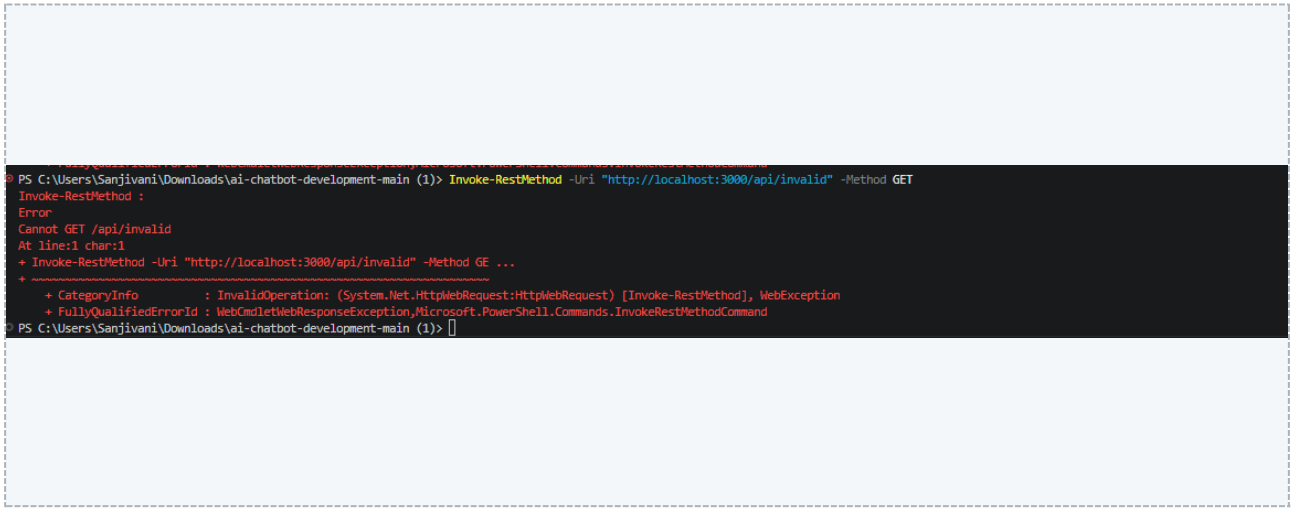

Figure 6: Invalid endpoint test result

8. Error Handling

The application includes basic error handling for different situations.

8.1 Empty Prompt

If a user submits a request without a prompt, the server returns:

```
{
  "error": "Prompt is required"
}
```

8.2 AI Model Error

If the AI model fails to generate a response, the server returns an error response.

8.3 Frontend Connection Error

If the frontend cannot connect to the backend server, an error message is displayed to the user.

8.4 Invalid Endpoint

Requests to unsupported routes return an error response from the backend server.

9. Project Directory Structure

The project repository is organised into separate folders for source code, data, reports, and deployment documentation.

```
ai-chatbot-development/
|-- data/
|   |-- chatbot_dataset.csv
```

```
|   `-- README.md
|-- src/
|   |-- index.html
|   |-- style.css
|   |-- script.js
|   |-- server.js
|   `-- README.md
|-- reports/
|   |-- AI Client Server Architecture.md
|   |-- API Specification.md
|   |-- API_Testing_report.md
|   |-- Task2_Data_Cleaning_Report.md
|   |-- data_schema.md
|   |-- deployment_report.md
|   `-- technical report.md
|-- deployment/
|   `-- README.md
|-- .gitignore
|-- package.json
`-- package-lock.json
```

10. GitHub and Version Control

Git and GitHub were used for source code management and version control. GitHub was used for:

- Storing project source code.
- Tracking project changes.
- Maintaining project documentation.
- Managing project files.
- Maintaining different versions of the project.

Important Git commands used during development included:

```
git add
git commit
git push
git status
```

The project repository is hosted on GitHub at:

github.com/sanjivanijadhav9116/ai-chatbot-development

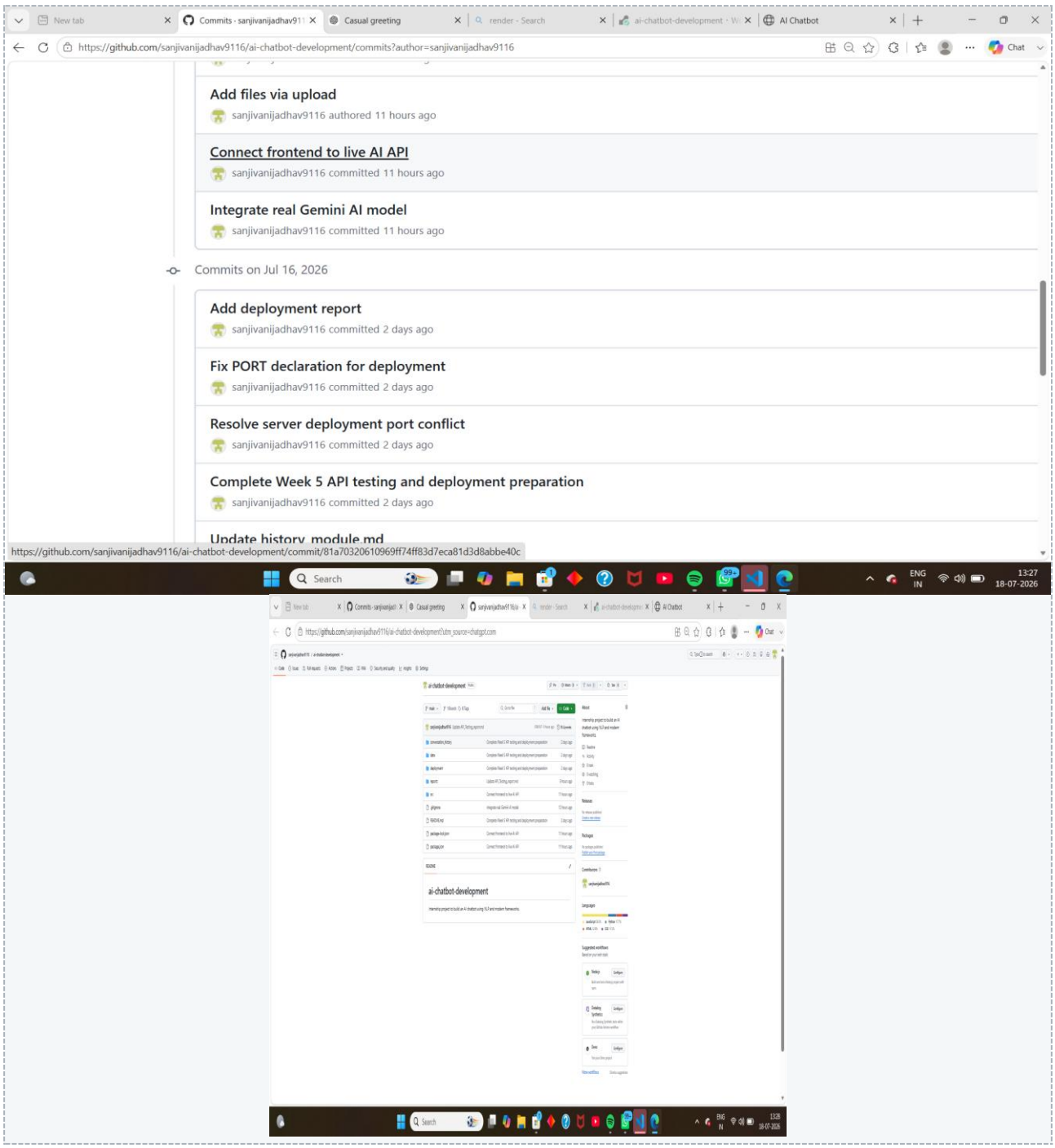

Figure 7: GitHub repository showing project files and commit history

11. Deployment

The backend application was deployed using Render. The deployment workflow was:

```
Local Development -> Git Repository -> GitHub Repository
  -> Render Web Service -> Live Backend API
```

Deployment Platform: Render

Build Command:

```
npm install
```

Start Command:

```
npm run start
```

Port Configuration — the application uses the environment-provided port:

```
const PORT = process.env.PORT || 3000;
```

This allows the server to run correctly both locally and in the cloud deployment environment.

The deployed service provides a live backend API for the chatbot application at:

ai-chatbot-development-lwt9.onrender.com

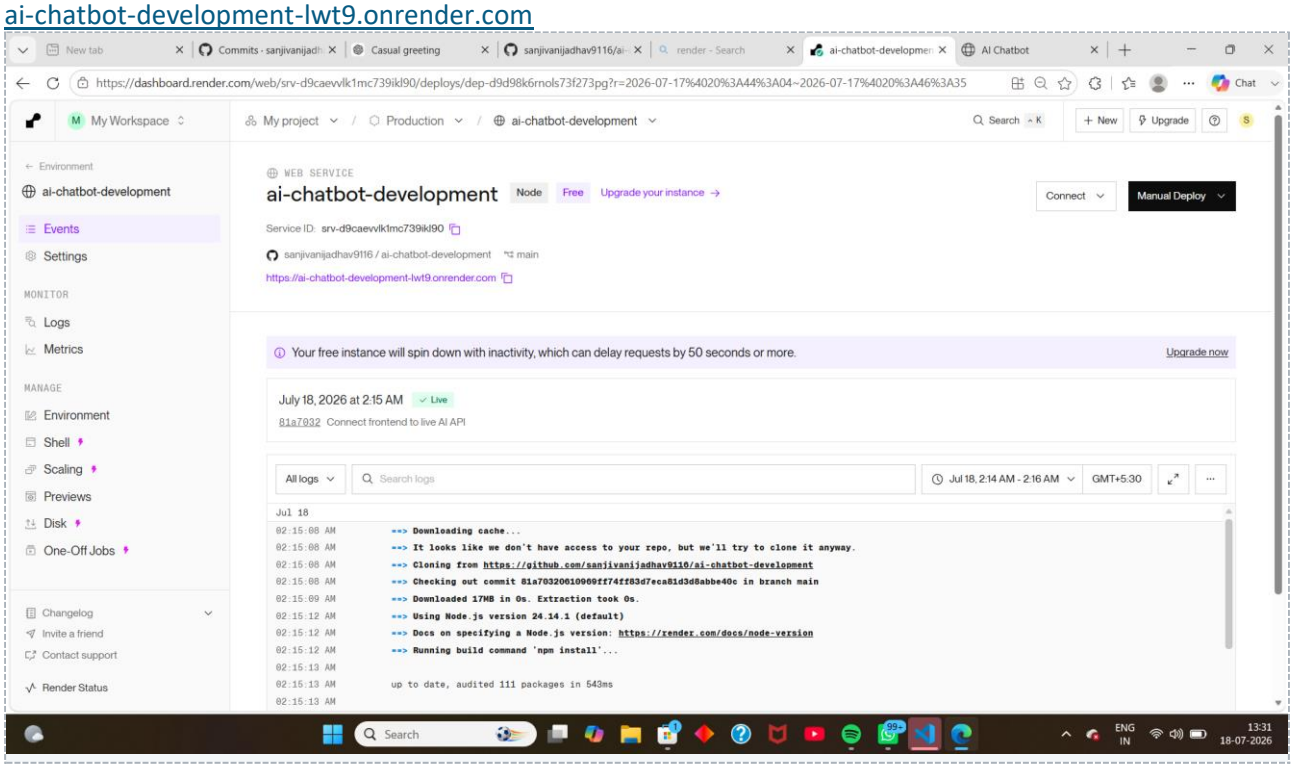

Figure 8: Render deployment dashboard showing the deployed web service

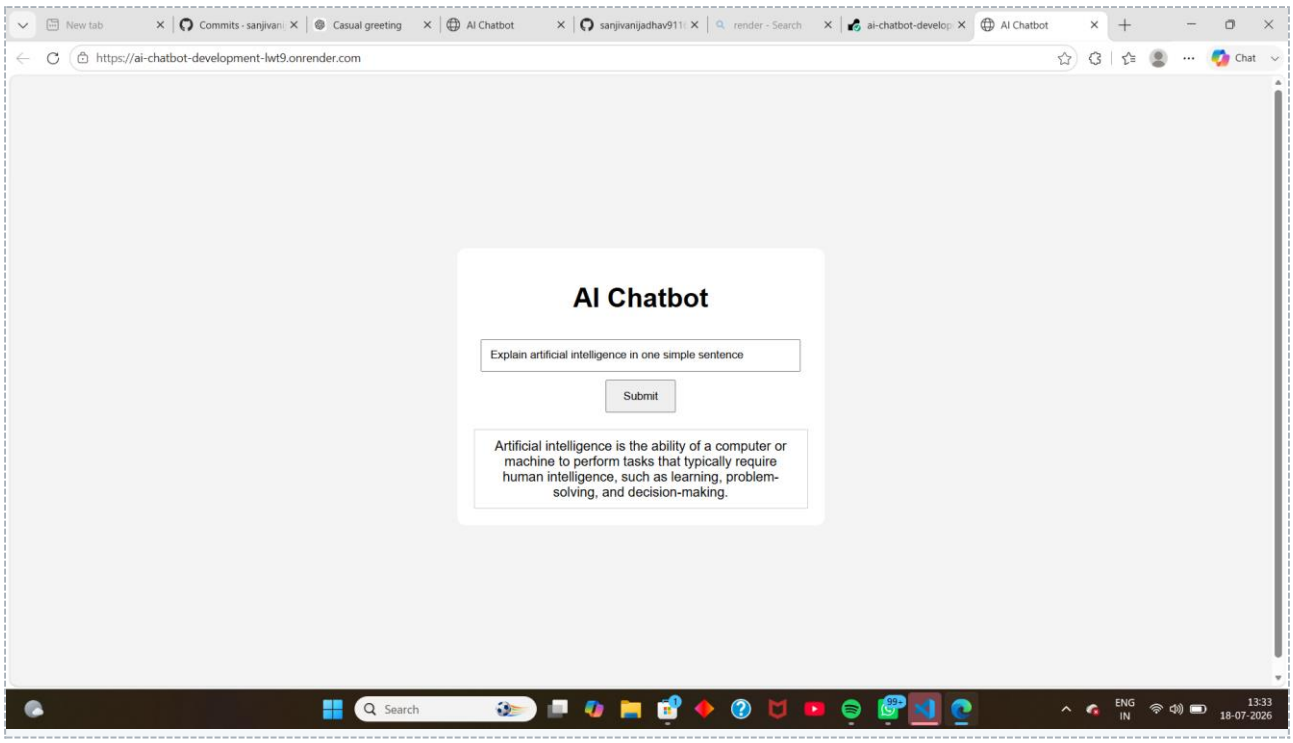

Figure 9: Live chatbot application running in the browser

12. Challenges Faced

Several technical challenges were encountered during the development process.

12.1 AI Model Integration

The project required selecting a compatible Gemini AI model and integrating it correctly with the Google GenAI SDK.

12.2 API Integration

Frontend and backend communication had to be configured correctly using the Fetch API and Express.js.

12.3 Input Validation

The backend required validation to ensure that empty prompts were rejected before being sent to the AI model.

12.4 Deployment Port Configuration

The server had to be configured to use the environment-provided port for successful cloud deployment.

12.5 API Testing

Different requests, responses, status codes, and error conditions were tested to verify backend functionality.

12.6 Version Control

Git and GitHub were used to track project changes and maintain the project repository.

13. Project Results

The project successfully achieved the following:

- Developed a functional chatbot interface.
- Implemented a Node.js and Express.js backend.
- Integrated the Google Gemini AI model.
- Implemented the /api/chat endpoint.
- Implemented supporting API endpoints.
- Implemented input validation.
- Tested the backend API functionality.
- Verified HTTP status codes.
- Connected the frontend with the backend.
- Successfully generated AI responses.
- Maintained source code and documentation using GitHub.
- Deployed the backend using Render.

14. Future Scope

The project can be further enhanced by adding the following features:

- Persistent conversation history.
- Database integration.
- User authentication.
- User profile management.
- Persistent feedback storage.
- Streaming AI responses.
- Multiple AI model support.
- Voice input and output.
- Multilingual chatbot support.
- Mobile application support.

15. Conclusion

The AI Chatbot Development Project successfully demonstrates the development of an AI-powered web application using a client-server architecture.

The system allows users to enter prompts through a frontend interface. The prompts are sent to a Node.js and Express.js backend, which communicates with the Google Gemini AI model and returns the generated responses to the user.

The project provided practical experience in frontend development, backend API development, client-server communication, AI model integration, API testing, error handling, version control, documentation, and cloud deployment.

Overall, the project demonstrates the complete development and deployment workflow of a functional AI chatbot application using modern web technologies and cloud deployment tools.

16. References

- Google AI for Developers — Gemini API Documentation, https://ai.google.dev/gemini-api/docs
- Node.js Documentation, https://nodejs.org/en/docs
- Express.js Documentation, https://expressjs.com
- Render Documentation, https://render.com/docs
- Git Documentation, https://git-scm.com/doc
- GitHub Docs, https://docs.github.com
- MDN Web Docs — Fetch API, https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API